

Face Identification on a microcontroller

Two neural networks recognise a face in under a second, on a 240 MHz chip, without any cloud.

BOARD

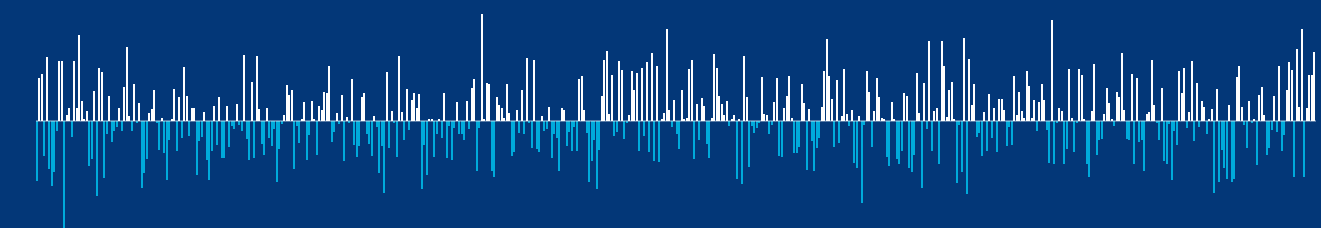
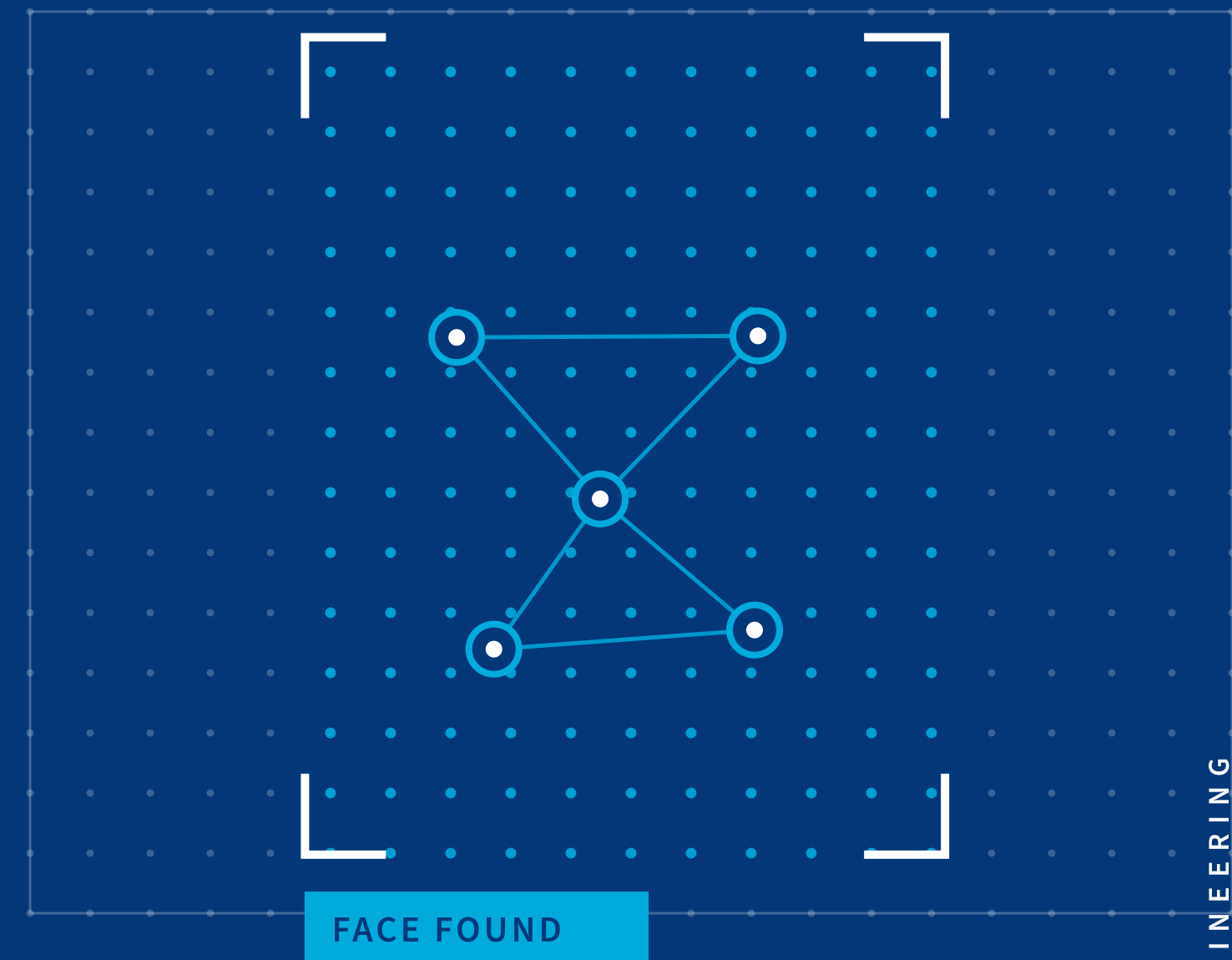
M5Stack CoreS3 Lite

CHIP

ESP32-S3, 2 × 240 MHz

SOFTWARE

Rust, no operating system



The test photo's face as the board describes it: 512 numbers

THE IDEA

Who's there? The board answers by itself.



The board's real screen with the detected face, its five landmarks and the result; captured with build faceid-14, before the recognizer switch (slide 15).

The app recognises up to four enrolled people with the board's own camera and chip. **No cloud, no laptop, no internet connection.**

HOW IT WORKS

- 1

Enrol

Tap ENROLL and turn your head a little: the board records six samples of your face.
- 2

Recognise

Look at the camera: in most cases the name appears at the first look. A stranger gets UNKNOWN.
- 3

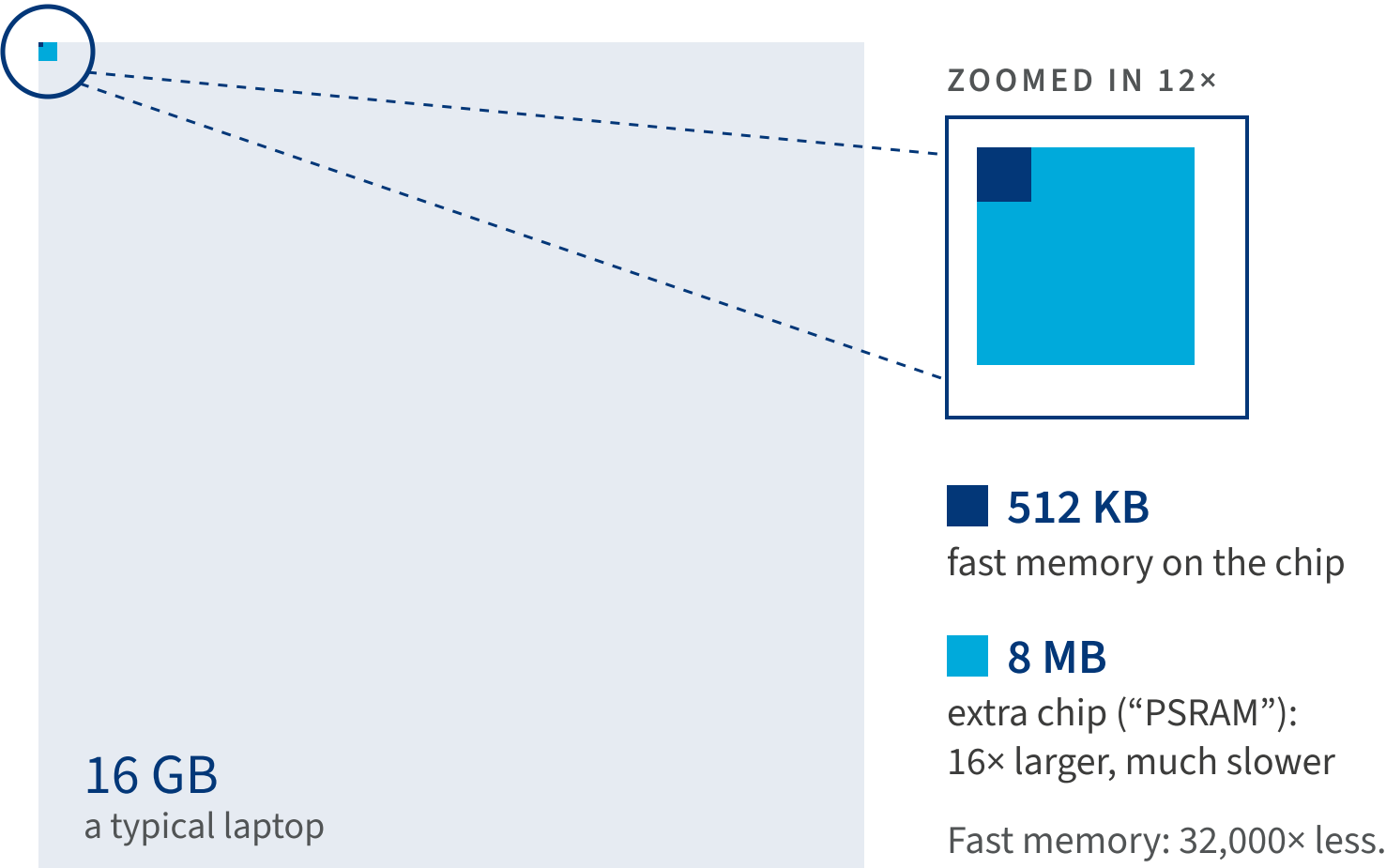
Remember

Up to four people, kept in the board's flash memory through restarts and updates.

THE CHALLENGE

A microcontroller is not a laptop

MEMORY, DRAWN TO SCALE



Our two face models are small by design, made for phones and small chips. Even so, the board has only a fraction of a laptop’s memory and speed, and **no operating system at all**.

OUR TWO MODELS, SMALL BY DESIGN

	TYPICAL LAPTOP	OUR BOARD
Processor	8+ cores at 3–4 GHz	2 cores at 240 MHz
Maths hardware	A graphics card with thousands of multipliers	One small vector unit: 16 small multiplications at a time
Floating point	float and double, both in hardware	float only; double is emulated in slow software

8.8 S

The first try on the board, for the recognizer alone (our first one, EdgeFace-XXS).
Goal: **under one second from face to name**.

01

Architecture

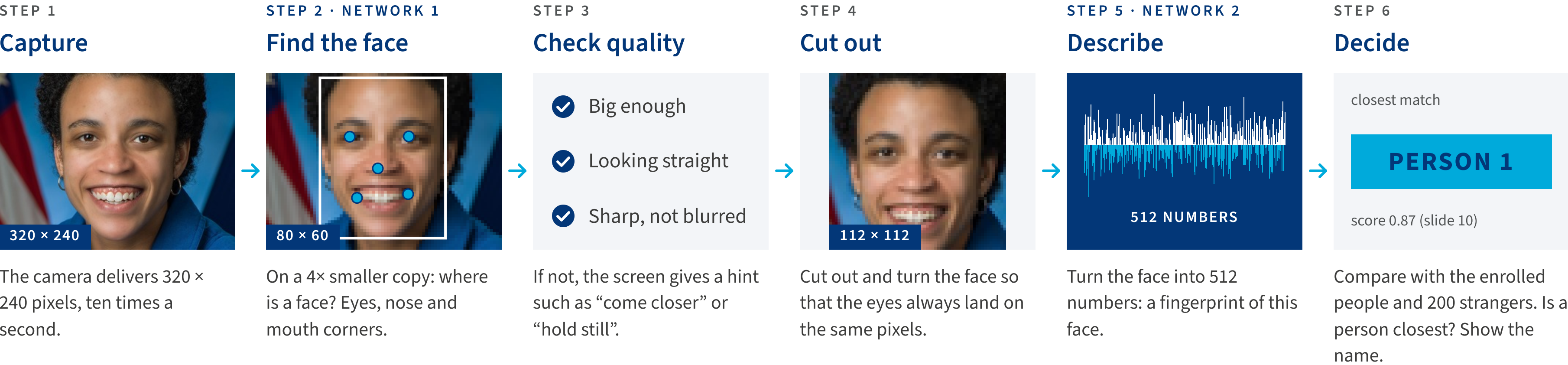
How do a camera, a small chip and two neural networks work together, and what does the laptop do?

- 05 Six steps from camera image to name
- 06 The laptop prepares, the board performs
- 07 Two cores, three kinds of memory

01 · ARCHITECTURE

Six steps from camera image to name

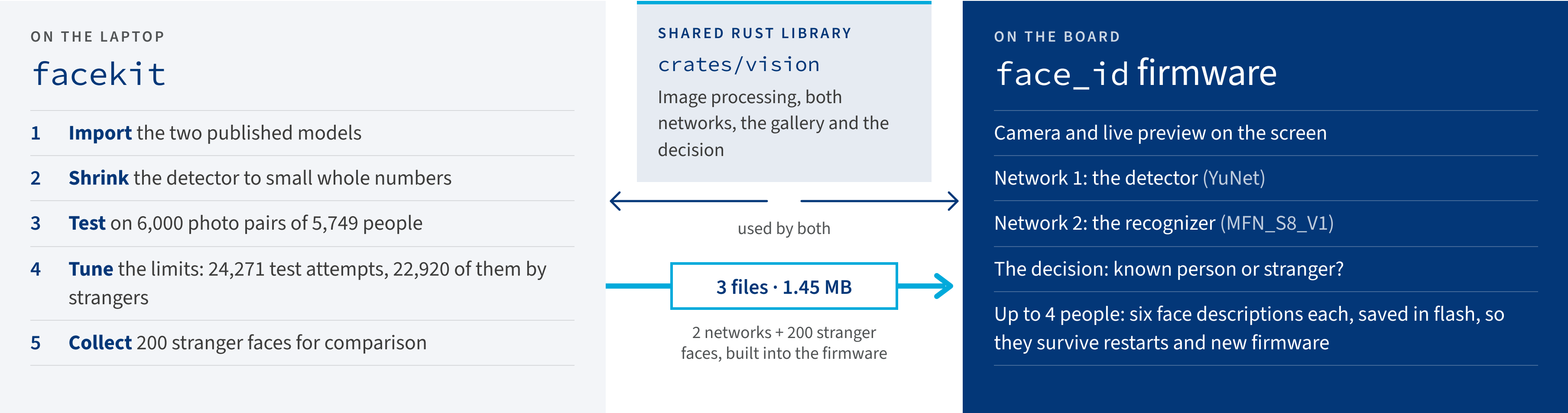
Every cycle runs the same chain. Two steps are neural networks; the others are classic image processing and bookkeeping.



01 · ARCHITECTURE

The laptop prepares, the board performs

A Rust toolkit on the laptop converts, tests and tunes the models. The board only runs them. Both use the same library, so both compute the very same numbers.



✓ **Tested on both machines:** at every start the board runs both networks on test inputs and compares the result with the laptop's. It matches bit for bit.

01 · ARCHITECTURE

Two cores, three kinds of memory

One core runs face identification, the other keeps touch, sensors, sound and USB alive. **Every piece of data lives in the fastest memory it fits into.**

ESP32-S3 chip

2 CORES · 240 MHZ

CPU0

Runs face identification

Main loop

find → check → cut out → describe → decide, again and again

Stream task

pauses the main loop briefly every 2 ms to collect the camera's pixels, and draws the live preview

CPU1

Keeps the rest alive

Touch screen

Motion and light sensors

Microphone, speaker

Radio (unused by this app)

USB log to the laptop

Internal RAM

512 KB, 144 KB free for data

fastest: a few clock ticks per read

WHAT LIVES HERE

The recognizer's work in bands: seven rows of a block's widest values at a time (69 KB), so they never go out to the slower PSRAM chip

PSRAM

extra chip · 8 MB · 40 MB/s

medium

WHAT LIVES HERE

The networks' learned numbers (1.35 MB), their working memory (0.8 MB) and the camera images

Flash

4 MB · 10 MB/s

slow, but keeps data without power

WHAT LIVES HERE

The firmware with both models built in, and the enrolled faces (128 KB)

02

Neural networks on a microcontroller

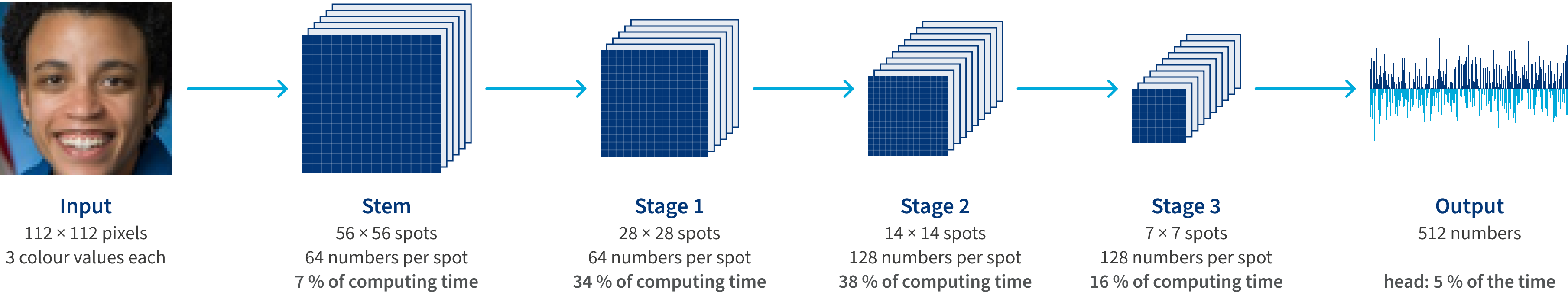
How do 1.19 million learned numbers fit onto a chip with 512 KB of fast memory, and run fast?

- 09 A neural network: multiply, add, repeat
- 10 A face becomes 512 numbers
- 11 Smaller numbers, same answers
- 12 The chip's secret weapon: the vector unit

02 · NEURAL NETWORKS

A neural network: multiply, add, repeat

No magic inside: the recognizer is **1.19 million numbers (“weights”)** that its authors learned from photos. Running it means multiplying values by weights and adding them up, layer after layer.



ONE MULTIPLY-AND-ADD, WORKED OUT

$(0.8) \times 0.4 + (0.2) \times (-1.2) + (0.5) \times 0.9 = 0.53 \rightarrow \text{next layer}$

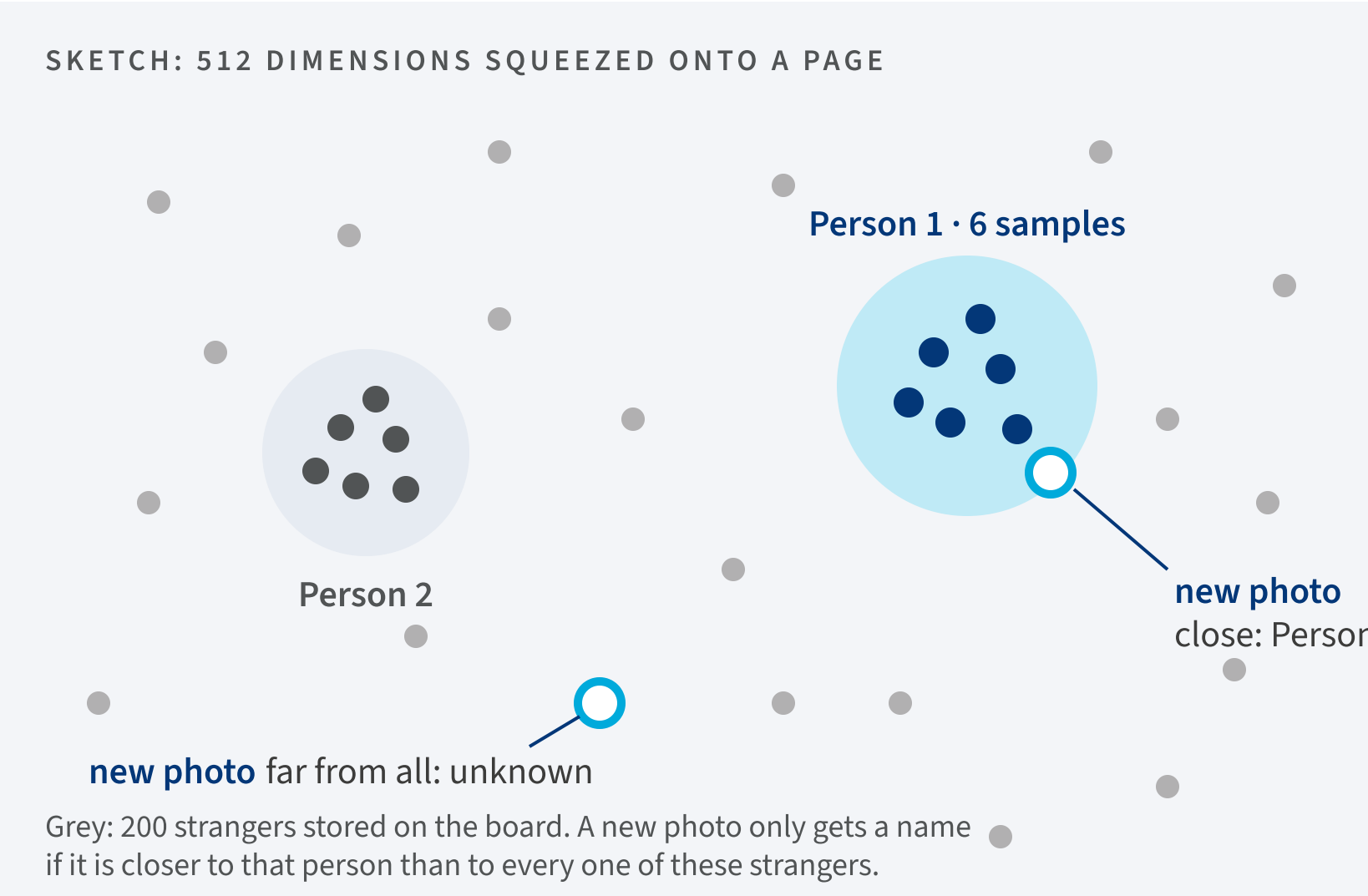
Values from the picture (circles) × learned weights, added up.
A layer does this for every spot: 221 million times per face.

Layer	One round of multiply-and-add like the one on the left, for every spot of the picture.
Block	Three layers as a team: widen, look at the neighbours, narrow.
Shrinking	The stem turns each 2 × 2 patch of pixels into one spot; each stage’s first block merges 2 × 2 spots into one.

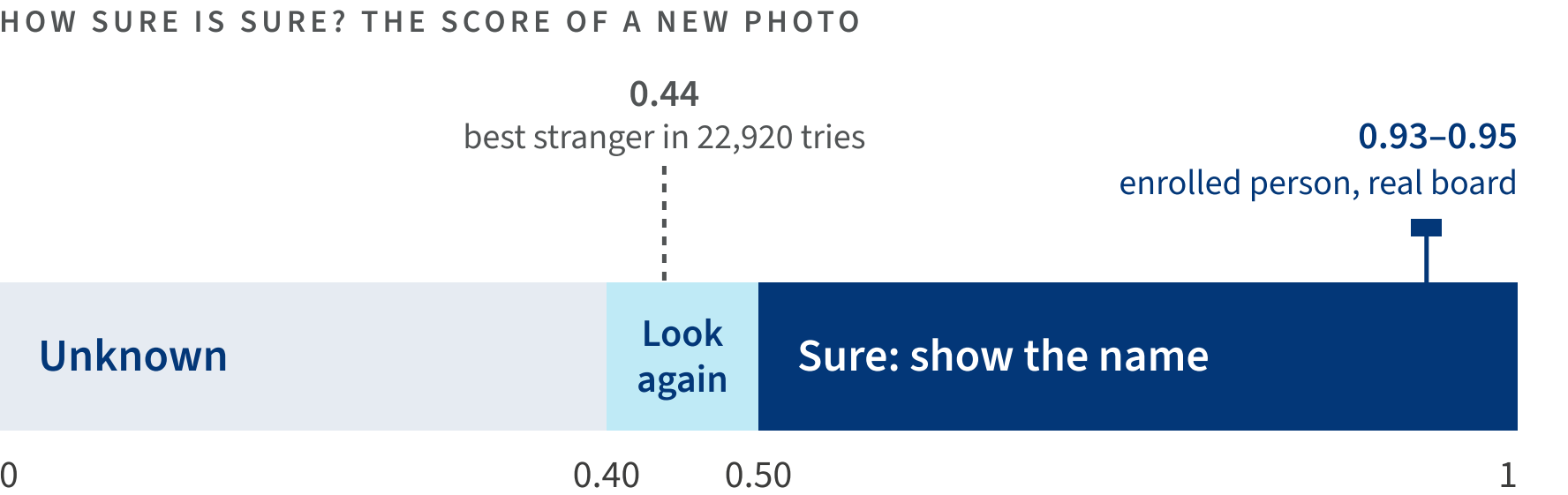
Percentages: each part’s share of the recognizer’s time for one face (434 ms, traced on the board). The detector (network 1, 53,121 weights) works the same way.

02 · NEURAL NETWORKS

A face becomes 512 numbers



The recognizer was trained so that two photos of the same person give nearly the same 512 numbers. **Comparing faces becomes comparing lists of numbers:** the score says how alike two lists are, from 0 to 1.



A name needs a score of at least 0.40, and higher than for any of the 200 stored strangers. Below 0.50 the board averages up to three looks. A stranger who looks enough like an enrolled person can still pass.

98.7 %

of simulated visits by enrolled people named at the first look, all of them by the second

5 of 22,920

stranger photos got a name: about 1 in 4,600. Fine for a greeting, not for a door lock.

How we tested: on the laptop, with public photos (LFW) and the board’s own decision code. 120 rounds: enrol one person from five photos, then show photos of that person and of 191 strangers.

02 · NEURAL NETWORKS

Smaller numbers, same answers

THREE KINDS OF NUMBERS (ONE BOX = ONE BIT)

32-bit float
how networks are trained



16-bit whole
the detector's values



8-bit whole
all of the recognizer



SIZE OF ALL RECOGNIZER WEIGHTS



More than the 512 KB of fast memory, so the weights live in PSRAM.

Same answers: our firmware code
against an independent interpreter of
Espressif's file

7,701 of 7,701 LFW photos
bit for bit

Networks are usually trained with 32-bit floating-point numbers, but the chip is much faster with small whole numbers. Espressif publishes MFN_S8_V1 **already in 8-bit whole numbers**, every tensor with one power-of-two scale. Our job: compute exactly what its file says.

ONE SCALE PER TENSOR, A POWER OF TWO

$52 \times 2^{-6} = 0.8125$ a whole number, in units of 1/64

Rescaling a sum is then a shift of its bits: no division, no float. The camera pixels go in the same way: $(x - 127.5) / 127.5$, in units of 2^{-6} .

BEFORE, WITH OUR FIRST RECOGNIZER, EDGEFACE-XXS

It came in 32 bits and we shrank it ourselves. **Try 1, 8 bits everywhere, failed:** small rounding errors piled up block after block, like copying a copy of a copy (closeness to the original 0.95 on average, worst 0.89). **Try 2** kept 16 bits for the values between layers: 0.9972 on average, worst 0.9955.

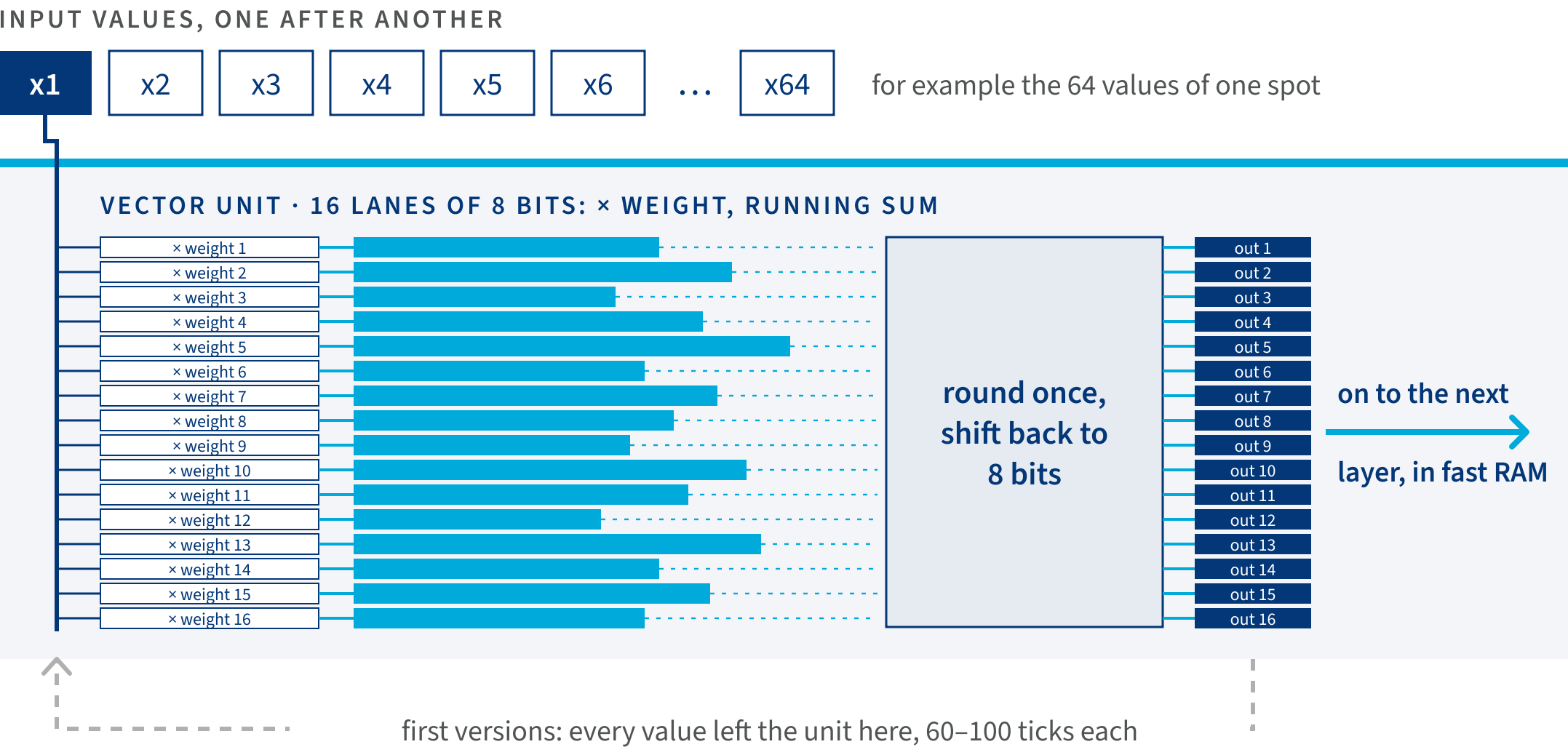
MFN_S8_V1 comes in 8 bits from Espressif, so there was nothing to shrink, only to reproduce.

The interpreter is part of our laptop toolkit and shares no code with the firmware. Accuracy on LFW: 99.27 % (slide 15).

02 · NEURAL NETWORKS

The chip’s secret weapon: the vector unit

Beside its normal core, the chip has a small vector unit: 8 “lanes” for 16-bit numbers, or **16 lanes for 8-bit numbers, side by side in one instruction**. The maths was cheap. What cost time was carrying values out to memory and back.



Why round at all? The values are small whole numbers (slide 11), so the big sums must be scaled back down.

CLOCK TICKS PER MULTIPLICATION

0.40

in all of the recognizer’s 1×1 layers, loading the numbers included

60–100

ticks for one value’s trip out to memory and back, in plain code

The first versions sent every result out of the unit after each layer. **Moving data took far longer than the multiplying.** Now results are rounded inside the unit, and each block works on bands of seven rows that stay in the fast internal RAM: the 14 × 14 block takes 41 ms row by row, 31 ms through PSRAM, **21 ms in bands of seven.**

WHAT THE 8-BIT MODE SAVED

~600 → 345 ms

All 1×1 layers of the recognizer: estimated on the 16-bit lanes, measured in the 8-bit mode. Same results, bit for bit.

03

Performance

The first try took 8.8 seconds per face. How fast could we make it, what did it take, and what did a new recognizer change?

- 14 From the first try to 0.39 s, step by step
- 15 A new recognizer, for the licence
- 16 Measure, don't guess: the chip's price list
- 17 The app around the networks
- 18 Rust without an operating system

03 · PERFORMANCE

23× faster, one measured step at a time

With our first recognizer, EdgeFace-XXS: nine measured steps, and the results stayed bit-for-bit identical after each. **The biggest single win was the most boring one:** 32-bit sums instead of 64-bit (step 02) saved 3.7 s, 44 % of the whole gain.

EDGEFACE-XXS, ONE FACE, ALONE			MS	SAVED
01		First version: straightforward code	8,842	
02	MATHS	Sum in 32 bits: no fast 64-bit multiply on this chip	5,113	−3,729
03	MEMORY	Use every value read from memory four times	3,724	−1,389
04	LANES	Use the 8 lanes: 8 multiplications at once	2,333	−1,391
05	MATHS	No slow divisions; weights in cache-sized chunks	1,560	−773
06	COMPILER	Fix what the compiler produced	1,418	−142
07	LANES	Store weights in the order the lanes read them	1,006	−412
08	LANES	Keep values inside the lanes between layers	729	−277
09	ONCE	Prepare everything once, when the app starts	472	−257
10	MEMORY	Work in slices that fit the fastest memory, plus small fixes	388	−84

SAVED PER IDEA

MATHS

−4.5 s

LANES

−2.1 s

MEMORY

−1.5 s

ONCE

−0.26 s

COMPILER

−0.14 s

“Alone”: the network by itself on the core. The detector went the same way: 1,568 → 99 ms. Today’s recognizer, MFN_S8_V1, does more than twice the work and takes 432 ms alone (next slide).

03 · PERFORMANCE

A new recognizer, for the licence

EdgeFace-XXS’s weights are licensed CC BY-NC-SA 4.0: no commercial use. The project goes public under MIT or Apache-2.0, so **since build faceid-16 the board recognises with Espressif’s MFN_S8_V1 (MIT).**

	EDGEFACE-XXS, BEFORE	MFN_S8_V1, TODAY
Licence	CC BY-NC-SA 4.0	MIT
Weights	1.24 million, shrunk by us	1.19 million, published in 8 bits
Multiply-adds per face	less than half	221 million
LFW, 6,000 photo pairs	99.42 %	99.27 % (± 0.54)
Accepted at 1 false in 10,000	97.50 %	97.60 %
Named at the first look	95.5 %	98.7 % (new limits)
Recognizer alone, board	383–388 ms	432 ms
Cycle that recognises, app	850–903 ms	894–919 ms

More than twice the work for 11–13 % more time, thanks to the 8-bit mode of the vector unit (slide 12). Accuracy: within measurement noise.

FACE RECOGNIZERS WITH A PERMISSIVE LICENCE	
OpenCV SFace (Apache-2.0)	9.9 MB even in 8 bits
fal AuraFace	a ResNet100
dlib	a ResNet at 150 × 150
Espressif MFN_S8_V1 (MIT)	1.26 MB: fits the board

Espressif does not say which photos it was trained on.

WHAT THE SWITCH TOOK

- 1 Read Espressif’s own file format in our toolkit (their loader is closed), with an interpreter of its own as the reference
- 2 New 8-bit kernels: 16 multiplications per instruction
- 3 New limits: accept from 0.40 instead of 0.35, and a new bank of 200 strangers

03 · PERFORMANCE

Measure, don't guess: the chip's price list

MEASURED COST: ONE SQUARE = ONE CLOCK TICK

8 multiplications in the vector unit (the instruction alone)



A simple instruction



A jump back to the start of a loop



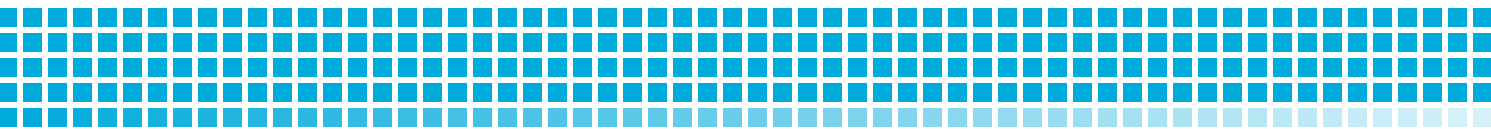
Reading a result out of the vector unit



One value through plain code between layers



A double (64-bit float), square root or logarithm



Our estimates from reading the source code were **2 to 4 times too optimistic, every time**. So we timed each operation on the board and read the machine code the compiler produced.

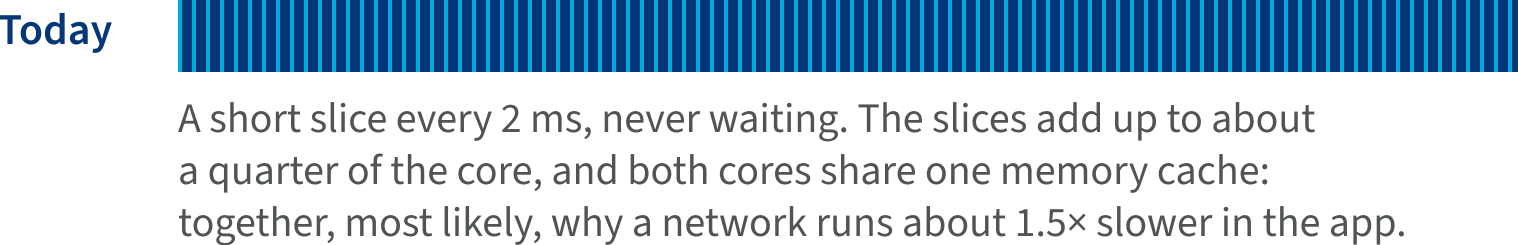
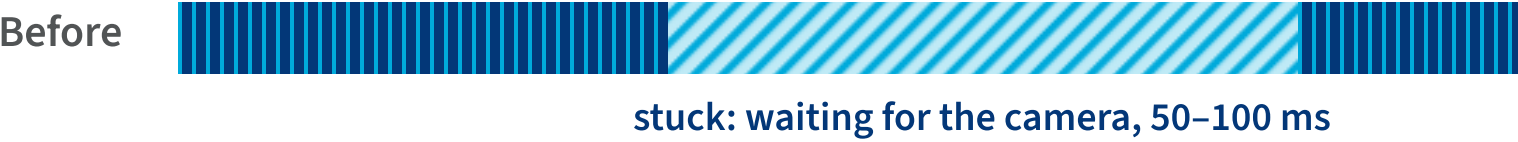
SURPRISES FOUND IN THE MACHINE CODE	
1	The compiler moved a slow 64-bit conversion into the loop, so it ran for every single output value
~1	
~3	512 bytes of memory were cleared before every batch of work, though nothing read them
~10	Results were read out one byte at a time: 200 instructions for 40 bytes
60-100	A helper we asked the compiler to inline became a real function call, once per value
hundreds	
Fixing these, plus packing the weights (EdgeFace-XXS):	
1,560 → 1,006 ms	

Memory has a price, too: the PSRAM chip delivers about 40 MB/s, flash about 10 MB/s. A 3 × 3 depthwise layer costs 1.15 ticks per multiplication from internal RAM, 3.24 from PSRAM.

03 · PERFORMANCE

The app around the networks

CORE 0 OVER 200 MS



■ networks, main loop ■ stream task: camera, screen

The camera has almost no buffer: its pixels must be collected within about 5 ms or they are lost. A network keeps the core busy for hundreds of ms. **Juggling both is like never blocking a UI thread.**

HOW WE FOUND THE STALLS

- 1 The log showed 256 lost camera images in 81 s, and a third of all time gone.
- 2 At the same moments, the motion sensor on the *other* core reported gaps. Why would it care?
- 3 Core 0 also runs the clock that both cores rely on. The camera code waited for the camera at a moment when nothing else may run on core 0, not even that clock, so both cores froze. **The fix: the camera code never waits; it comes back 2 ms later.**

1,730 → 862 ms	505 → 173 ms
a cycle that recognises a face	a cycle without a face

First app build whose cycle was logged, against faceid-13 (EdgeFace-XXS). With MFN_S8_V1 today: 894–919 ms and 144–195 ms.

03 · PERFORMANCE

Rust without an operating system

WHAT RUST GAVE US

One library, two machines

The same Rust code runs in the laptop's tests and on the chip. That is what made the bit-for-bit check possible.

async/await, without an OS

Camera and screen run as an async task that a timer wakes every 2 ms. A small async runtime does it all.

Hardware as owned values

The app owns its camera, screen and touch handles. No other code can use them by accident.

ON THE LAPTOP: TEST THE SHARED LIBRARY

```
./scripts/test.sh -p hack-and-hike-vision
```

No operating system, no threads, no files: just our Rust code on the chip. **What the language gave us, and where it pushed back.**

WHERE IT PUSHED BACK

The fast maths needs assembly

The vector unit (slide 12) is only reachable through hand-written assembly inside the Rust code.

A compiler of its own

The chip needs its vendor's own Rust toolchain, and the strongest optimisation setting makes that compiler crash.

64-bit floats are slow here

Rust compiles them without complaint, but this chip has no hardware for them, so every calculation runs as slow software. One in the wrong place cost 32 ms per block. We use 32-bit floats.

FOR THE BOARD: BUILD THE APP ON TOP OF IT

```
cargo dist --bin face_id
```

04

Results

Speed measured on the board. Accuracy measured on the laptop with the identical integer maths, which the board reproduces bit for bit.

0.43 s

recognizer, one face
first try 8.8 s, with EdgeFace-XXS;
the detector 1.6 s → 0.1 s

0.9 s

from face to name
in the full app (894–919 ms), named
at the first look

99.27 %

correct on LFW
6,000 photo pairs; bit for bit what
the board computes

98.7 %

named at the first look
in simulated visits; on the board,
too

1:4,600

strangers let in
5 of 22,920 simulated attempts

0

servers needed
everything runs on the board,
offline

Limits, honestly: no liveness check, so a good photo may fool it · up to 4 people · tuned on public photos, not on our office light · a greeting, not a door lock.

04 · WHAT WE TAKE AWAY

Four lessons for the next team

- 1

Measure on the device

Our estimates were 2–4× off. The board’s own numbers never were.
- 2

Moving data costs more than maths

Keep values in fast memory and inside the vector unit.
- 3

Prove every change

A bit-for-bit check against the laptop made bold changes safe.
- 4

Read the model’s licence first

Ours was non-commercial. Swapping it cost a new set of kernels.

Next: put the second core to work. Its helper tasks leave it mostly idle, and sharing each layer between both cores could almost halve the time again. In the recognizer, two blocks (57 and 52 ms) and the stem (32 ms) still have room.

WHY IT MATTERS BEYOND THIS DEMO

- Private by design:

faces are recognised on the device, not in a cloud.
- Small, low-power hardware

instead of a PC with a graphics card.
- The same recipe

fits other on-device AI: machine vision, sensors, medical devices.

PICTURE THIS

A lab instrument that recognises its operator and loads their settings. Offline, and no photo ever leaves the device.

“Measure, don’t guess.”

The rule behind every step from 8.8 s to under half a second.

Thank you. Questions?

Swiss Software Engineering
better ask ERNI